LLM: 사전학습, 프롬프팅, 정렬

언어모델링: 다음 토큰 예측과 사전학습·파인튜닝 · 프롬프팅 · 포스트트레이닝: SFT, RLHF, DPO

Machine Learning 1 · 17주차

한국과학영재학교 (KSA)

Week 17

이번 주차 개요

- 의대생은 해부학·생리학·약리학 같은 **방대한 기초**를 먼저 익히고 나서야 특정 전공을 집중 훈련한다. LLM도 비슷: 방대한 텍스트로 다음 단어 예측을 익혀(**사전학습**) 폭넓은 지식을 흡수하고, 적은 데이터로 다듬고(**파인튜닝**), "사람이 실제로 원하는 방향"으로 정교하게 다듬는다 (**포스트트레이닝**).
- **17.1 언어모델링** — "다음 토큰 예측"이 **문장 전체 분포 학습과 동일한 과제**임을 사슬 분해로 보이고, BPE·softmax·온도·교차 엔트로피·PPL을 작은 예제로 확인한 뒤 사전학습/파인튜닝의 현대 도식으로 마무리.
- **17.2 프롬프팅** — 가중치 없이 **입력만으로** 행동을 바꾸기: zero-shot → few-shot → CoT 스펙트럼, temperature·top-k·top-p, 그리고 환각.
- **17.3 포스트트레이닝** — InstructGPT의 관찰에서 출발해 SFT → RLHF(보상모델 + PPO) → DPO를 짚고, LoRA/PEFT와 RAG·에이전트까지 모은 전체 지도.

이 장을 마치면 (1∼16장 이후의 심화·응용 부록)

- softmax·교차 엔트로피·퍼플렉시티를 작은 예제로 직접 계산, BPE가 "단어 조각" 어휘를 만드는 이유를 설명할 수 있다.
- SFT·RLHF·DPO를 그림으로 구분하고, "그럴듯한 텍스트"가 "사람이 원하는 답변"과 왜 다른지 설명할 수 있다.

17.1 언어모델링: 다음 토큰 예측과 사전학습·파인튜닝

언어모델: “다음 토큰을 예측하는 함수”

- 다음 토큰의 확률분포를 예측
- decoder-only Transformer로 계산
- 퍼플렉시티(PPL) 정의

사슬 분해: 다음 토큰 예측 = 문장 전체 분포 학습

- 조건부 확률의 **사슬 분해**:

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, \ldots, x_{t-1})$$

다음 토큰 예측 = 문장 전체 분포 학습

토큰은 왜 “단어 조각”인가: BPE

- **BPE**(Byte Pair Encoding)가 이 모순의 표준 해결책:
  1. 토큰을 문자(한글 글자, 바이트) 단위로 쪼갬.
  2. “문서에서 가장 자주 함께 나타나는 두 인접 토큰”을 **반복 병합(merge)**.
  3. 어휘 사전이 목표 크기(예: 32,768개)가 될 때까지.

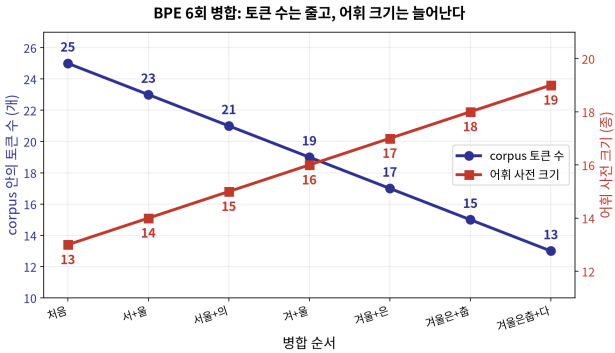
BPE 손계산: 세 줄 corpus에서 6회 병합

corpus: “서울의 겨울은 춥다 / 서울의 봄은 따뜻하다 / 파리의 겨울은 춥다” (여백 없이 문자 단위 → 25개 글자, 서로 다른 13종)

병합	병합 내용(빈도)	토큰 수	어휘 사전 크기
처음	각 문자	25	13
1	서+울 → 서울 (2회)	23	14
2	서울+의 → 서울의 (2회)	21	15
3	겨+울 → 겨울 (2회)	19	16
4	겨울+은 → 겨울은 (2회)	17	17
5	겨울은+춥 → 겨울은춥 (2회)	15	18
6	겨울은춥+다 → 겨울은춥다 (2회)	13	19

- **반복되는 조각이 먼저 묶인다**

그림으로: 병합이 진행될수록



병합이 반복될수록 corpus 안의 **토큰 수는 줄고** (25 → 13), **어휘 사전 크기는 늘어난다**(13 → 19).

손으로 한 번: n-gram으로 다음 토큰 분포 세기

- “다음 토큰의 확률”을 **앞의** $n-1$**개 토큰만으로 조건부**로 근사:

$$P(x_t \mid x_1, \ldots, x_{t-1}) \approx P(x_t \mid x_{t-n+1}, \ldots, x_{t-1})$$

- $n = 1$: 앞 토큰을 보지 않는 무조건분포(어휘 전체에 나눈 단어 빈도). $n = 2$: **bigram**.

bigram 모델: “서울의” 뒤에?

```
from collections import defaultdict
import math

def next_token_distribution(corpus_tokens, context_word):
    # 가context_word 등장한모든위치에서  , 바로다음토큰을
    # 다음단어{: 확률} 딕셔너리로반환  (bigram 모델, n=2)
    counts = defaultdict(int)
    for i in range(len(corpus_tokens) - 1):
        if corpus_tokens[i] == context_word:
            counts[corpus_tokens[i + 1]] += 1
    total = sum(counts.values())
    if total == 0:
        return {}   # 문맥이한번도등장한적없으면예측근거없음
    return {w: c / total for w, c in counts.items()}

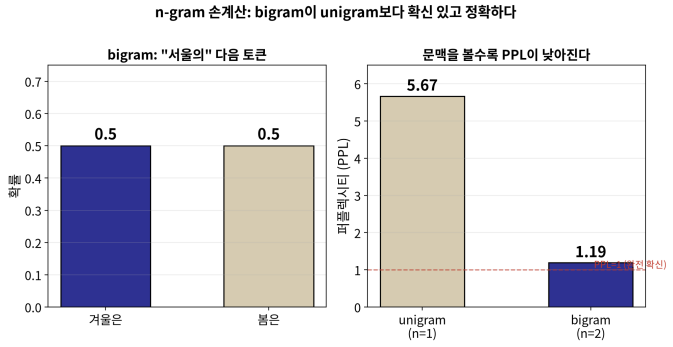
corpus = "서울의 겨울은춥다서울의봄은따뜻하다파리의겨울은춥다        ".split()
print(next_token_distribution(corpus, "서울의"))
# 겨울은{'': 0.5, 봄은'': 0.5}  -- 서울의"" 뒤에는겨울봄이 / 각 50%
```

n-gram의 두 한계, 그리고 PPL을 손으로

- **bigram($n = 2$)의 퍼플렉시티**
- **unigram ≈ 5.67 vs bigram ≈ 1.19**

좋은 언어모델 = PPL이 낮은 모델

그림으로: n-gram 손계산 결과



왼쪽: bigram이 “서울의” 다음에 준 분포(겨울은/봄은 각 50%). 오른쪽: 문맥을 볼수록 PPL이 5.67 → 1.19로 낮아진다.

softmax: 원점수를 확률로 — 숫자로

- **softmax: 원점수를 확률로**
- 숫자로: $z = (4, 1, 0, -3)$ 이면 $e^z \approx (54.6, 2.72, 1.0, 0.05)$, 합 $\approx 58.37 \to$ 확률 $\approx$ **(0.935, 0.047, 0.017, 0.001)** — 첫 토큰에 93.5%가 모인 “확신 있는 예측”.

온도(Temperature): 확신도를 조절하는 손

- softmax에 **온도** $T > 0$을 넣으면:

$$P(x_t = j) = \frac{e^{z_j/T}}{\sum_k e^{z_k/T}}$$

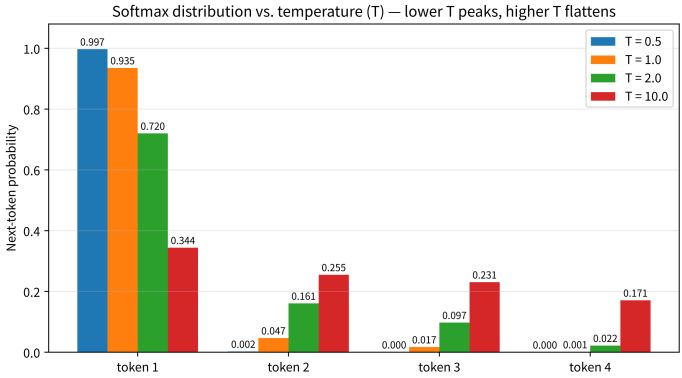
(원점수를 T로 나눠준 뒤 softmax)

- **T<1: 점근, T>1: 평탄**

실전 LLM 채팅봇의 temperature 파라미터

"창의적인 글" $\to T > 1$(확률을 넓혀 덜 예상적인 토큰도) / "사실·코드" $\to T \to 0$(가장 확신 있는 토큰만). **동일한 softmax 분포를, "토큰을 고를 방법"에 따라 확신도를 조절**하는 것 — 17.2절 샘플링 전략과 17.3절 포스트트레이닝(고온 샘플링으로 다양한 후보를 만들어 비교)의 기초.

온도의 그림



$T = 0.5, 1.0, 2.0, 10.0$에 따른 softmax 분포: T가 작을수록 확신이 한 토큰에 몰리고(점근), 커질수록 네 토큰에 고르게 퍼져(평탄) 무작위성이 커진다.

- “창의적”과 “정확”을 한 파라미터로 **연속적으로** 조절.
- 17.2절에서 이 분포 위에 **top-k, top-p** 샘플링 전략이 올라온다.

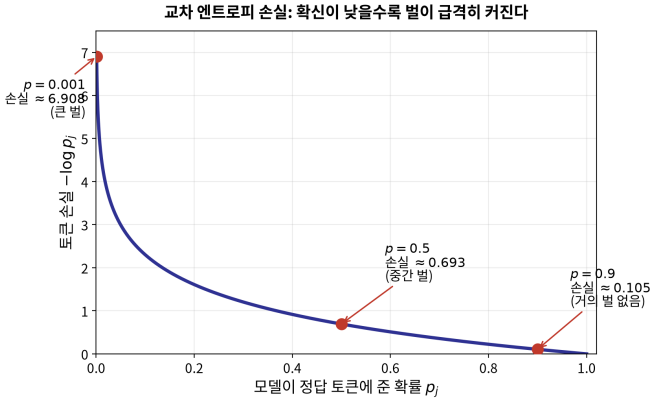
교차 엔트로피 손실: “정답이 받은 확률”을 최대화

- 모델이 토큰 j에 확률 p_j를 부여했고, **실제로 정답이** j였을 때 그 한 토큰의 손실:

$$\mathcal{L}_{\text{token}} = -\log p_j$$

- 세 가지 값으로 읽기: $p_j = 0.9 \to -\log 0.9 \approx 0.105$(거의 맞힘, 거의 벌 없음) / $p_j = 0.5 \to 0.693$(동전 던지기 수준, 중간 벌) / $p_j = 0.001 \to 6.908$(거의 0 확률에 정답, 큰 벌).

그림으로: 교차 엔트로피 손실 곡선



$-\log p_j$는 p_j가 0에 가까울수록 **폭발적으로** 커진다 — 정답에 낮은 확률을 준 실수는 큰 벌, 이미 높은 확률(0.9 이상)을 준 경우는 손실이 거의 평평해 그래디언트도 작아진다(안정적으로 수렴하는 이유).

문장 손실과 PPL: 하나의 사슬

- 문장(minibatch) 전체 손실은 토큰 손실의 **평균**:

$$\mathcal{L} = -\frac{1}{N}\sum_{t=1}^{N} \log P(x_t \mid x_1, \ldots, x_{t-1})$$

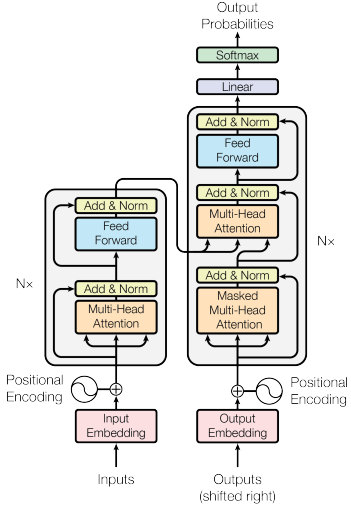
- **퍼플렉시티는 이 손실의 지수**:

$$\text{PPL} = e^{\mathcal{L}}$$

- $\mathcal{L} = 2.3$이면 PPL $= e^{2.3} \approx 10$ — “평균 10개의 균등한 후보 중 하나를 고르는 것과 같음”.

역사: n-gram에서 신경망까지

- n-gram의 한계: 문맥 길이와 OOV
- word2vec: 분포 가설 기반 임베딩
- Transformer: 자기-어텐션과 스케일



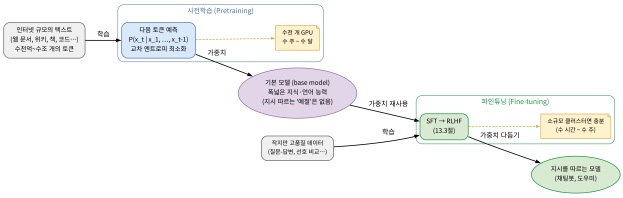
Vaswani et al. — “Attention Is All You Need” (2017), Figure 1: 2017년 이 자기-어텐션 구조 하나가, 80년 걸친 n-gram → word2vec의 역사를 LLM이라는 새 현상으로 밀어올렸다.

사전학습 vs 파인튜닝

	사전학습(Pretraining)	파인튜닝(Fine-tuning)
데이터	인터넷 규모의 방대한 텍스트	특정 목적에 맞는 비교적 적은 데이터
목표	다음 토큰 예측(비지도)	특정 작업(대화, 지시 따르기 등)에 맞춘 지도학습
결과	폭넓지만 다듬어지지 않은 능력	목적에 맞게 다듬어진 행동

- 사전학습: 계산 비용 **막대** — 수천 GPU, 몇 주∼몇 달. 파인튜닝: 상대적으로 적은 자원.

2단계 파이프라인과 비용의 비대칭성



인터넷 규모 텍스트로 다음 토큰 예측(수천 GPU, 수 주∼수 달) → 기본 모델 → 질문-답변/ 선호 데이터로 SFT·RLHF 파인튜닝(소규모 클러스터, 수 시간∼수 주) → 지시를 따르는 모델.

- 사전학습 비용은 **데이터의 양**(수조 토큰)에 비례 → **"한 번만"**.
- 파인튜닝 비용은 **작업의 수**(대화, 코드, 요약, 역할극…)에 비례 → **"작업마다"**.

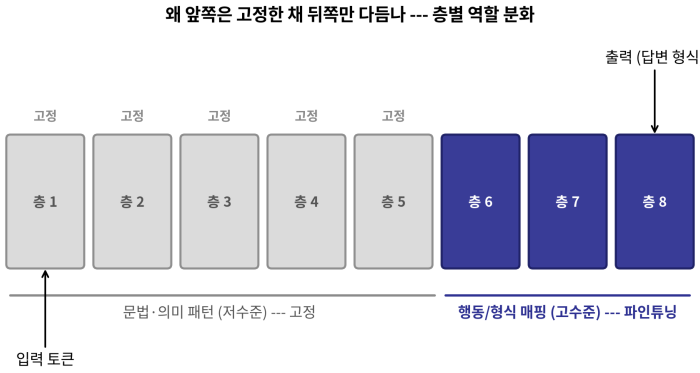
하나의 기본 모델 + 여러 파인튜닝

"여러 개의 별도 사전학습 모델"보다 효율적인 이유 = 이 **비용의 비대칭성**(데이터 비례 vs 작업 비례).

왜 앞쪽은 고정한 채 뒤쪽만 다듬나

- - **앞쪽 Transformer 층**: 토큰 주변의 문법/의미 패턴 — **문언어적(low-level linguistic) 특징**
  - **뒤쪽 층**: 어떤 내용/톤/형식의 답변이 적절할 것인가 — **고수준 의미/행동 특징**
- **파인튜닝은 뒤쪽 층 중심 행동 매핑**

그림으로: 층별 역할 분화



앞쪽 층은 문법·의미 같은 저수준 패턴을 이미 담고 있어 **고정**, 뒤쪽 층만 "어떤 상황에 어떤 답변"이라는 고수준 행동 매핑으로 **다듬는다** — Week 10.3 CNN 전이학습과 같은 논리.

자주 묻는 질문 (17.1)

- **Q. PPL이 낮으면 항상 더 똑똑한가?** PPL은 “다음 토큰을 얼마나 **확신 있게** 예측하는가”만 잰다 — 사실 정확성·추론은 **별도로** 평가. 같은 문장만 반복하는 데이터로 학습한 모델은 PPL이 매우 낮을 수 있지만 쓸모없다. **PPL과 작업 성능 벤치마크를 항상 함께 본다.**
- **Q. 데이터 양과 모델 크기의 관계?** **스케일링 법칙**(Kaplan et al., 2020): 파라미터 수· 데이터 양·계산량 중 하나만 키우면 금방 한계 — 셋을 **적절한 비율로 함께** 키워야 계속 좋아진다는 경험적 관계. “얼마나 크게, 얼마나 많이 모아야 하는가”의 실무 지침.
- **Q. “다음 토큰 예측”은 너무 단순한 것 아닌가?** 목표는 단순하지만 **분포가** 단순하지 않다. “1 + 1 = ?” 뒤에 “2”가 확실히 오는 분포를 만들려면 **덧셈의 규칙을 일반화**해야 한다 — **단순한 목표 + 복잡한 분포 = 일반화되는 능력**이 바로 **신흥 능력**(emergent ability, Wei et al., 2022)의 기원. 17.2절 CoT(Wei et al., 2022)가 이 능력을 **의식적으로 유도**.
- **Q. BPE 어휘 크기는 어떻게 정하나? 3만∼20만** 표준. 작으면 토큰이 길어져 **문맥 길이 제한**(Week 12 $O(n^2)$ 어텐션), 커지면 **임베딩 파라미터**($V \times d$; $V = 200{,}000, d = 4096$이면 약 8×10^8) 커짐 — **토큰 길이 vs 임베딩 비용** 트레이드오프. 언어/도메인에 따라 병합 순서가 달라져 **같은 문장도 모델마다 다른 토큰 수**로 측정.

자주 하는 실수 (17.1)

- **OOV 오해와 직전 단어 오해**
- **PPL과 작업 성능 혼동**

두 구분이 “왜” BPE와 Transformer를 쓰는지의 핵심

토큰 $\neq$ 단어, 문맥 = 앞의 모든 토큰. PPL = 언어의 그럴듯함, 벤치마크 = 작업의 정확성.

연습문제: softmax·CE·PPL을 완성하라

```python
import math

def softmax(z):
    # z: 원점수리스트  -> 확률분포합  ( 1)
    # 수치( 안정: max(z)를 먼저빼고지수를취하라   )

def cross_entropy_loss(probs, true_idx):
    # -> -log(probs[true_idx]) 반환

def perplexity(token_losses):
    # -> exp평균( 손실) 반환

# 검증본문 ( 예: z=(4,1,0,-3))
p = softmax([4.0, 1.0, 0.0, -3.0])
assert abs(p[0] - 0.935) < 1e-3        # [0.935, 0.047, 0.017, 0.001]
assert abs(cross_entropy_loss(p, 0) - 0.067) < 1e-3
print(f"{perplexity([0.067, 0.693, 3.912]):.2f}")  # e^1.557 ~= 4.75
```

실습 노트북: ① BPE 5회 병합(3개 토큰) ② bigram (0.5, 0.5) + PPL 5.67 vs 1.19 ③ softmax 검증 ④ 온도 4종 분포 ⑤ CE·PPL 세 값 ⑥ **미니 신경망 언어모델 학습** — 학습 곡선(손실 감소)으로 "다음 토큰 예측 = 분포 학습"을 수치로 확인.

17.2 프롬프팅

프롬프팅: 다시 학습시키지 않고 행동을 바꾸기

- **프롬프팅**(prompting)은 가중치를 **전혀** 바꾸지 않고 **입력(지시문, 예시)**만으로 출력을 원하는 방향으로 유도.
- 세 가지 기법:
  - **Zero-shot**: 예시 없이 지시문만 — “이 문장을 한국어로 번역해줘: …”
  - **Few-shot**: 입출력 예시 몇 개 포함시켜 패턴 파악 후 따라하게 (Brown et al., 2020).
  - **Chain-of-Thought(CoT)**: 최종 답 전에 **중간 추론 과정**을 출력하도록 유도.

스펙트럼: “프롬프트에 얼마나 보여줄 것인가”

zero-shot = “**무엇을** 할지”만 말하고, few-shot = “어떤 **형식**으로” 하는지 보여주고, CoT = 어떤 **과정**으로 하는지까지 보여준다. 공통: **가중치를 건드리지 않는다** — 학습 업데이트(그래디언트, 역전파)가 한 번도 일어나지 않고 **추론(inference) 시점**에 프롬프트만으로 동작이 바뀜.
파인튜닝 = “모델을 **바꾸는**” / 프롬프팅 = 그 자리에서, **즉각적으로, 원상복구 가능하게** “모델의 사용법을 바꾸는”.

왜 “예시 몇 개”가 새 작업을 가르치는가

- few-shot은 사전학습 능력의 증거
- 새 지식 학습이 아닌 능력 선택
- LLM 출력은 조건부 확률

장난감 모델로 확인: 예시가 있으면 더 잘 맞히는가

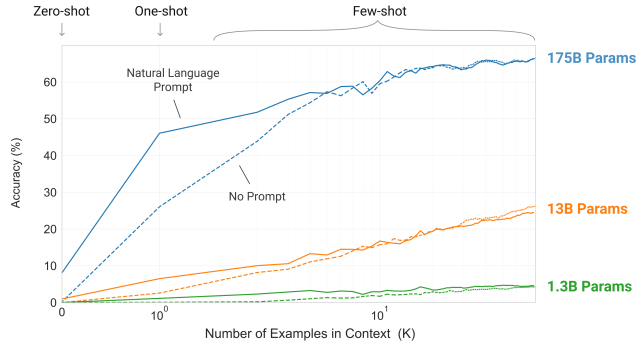
```
# zero-shot: 사과는이라는"" 패턴을프롬프트에서한번도보여주지않음
zero_shot_context = "오늘 기분이좋다사과는  ".split()
print(next_token_distribution(zero_shot_context, "사과는"))
# {}  -- 아무근거가없어예측불가

# few-shot: "는OO 이다OO" 패턴예시개  2 + 질문
few_shot_context = "사과는 과일이다바나나는과일이다자동차는기계이다사과는      ".split()
print(next_token_distribution(few_shot_context, "사과는"))
# 과일이다{'': 1.0}  -- 프롬프트안의예시가곧바로예측근거가됨

# 한계: 비행기는은"" 예시에서나온적없으니    , 패턴을알더라도  {}
```

프롬프트에 넣은 토큰이 곧바로 다음 예측의 조건이 된다

그림으로: 실제 GPT-3에서도 같은 현상



Brown et al. — GPT-3 (2020), Figure 1.2: 모델이 클수록 zero → one → few-shot의 성능 격차가 더 벌어진다 — 장난감 모델의 원리가 실제 GPT-3에서도 확인된다.

Few-shot 예시의 실무 요령

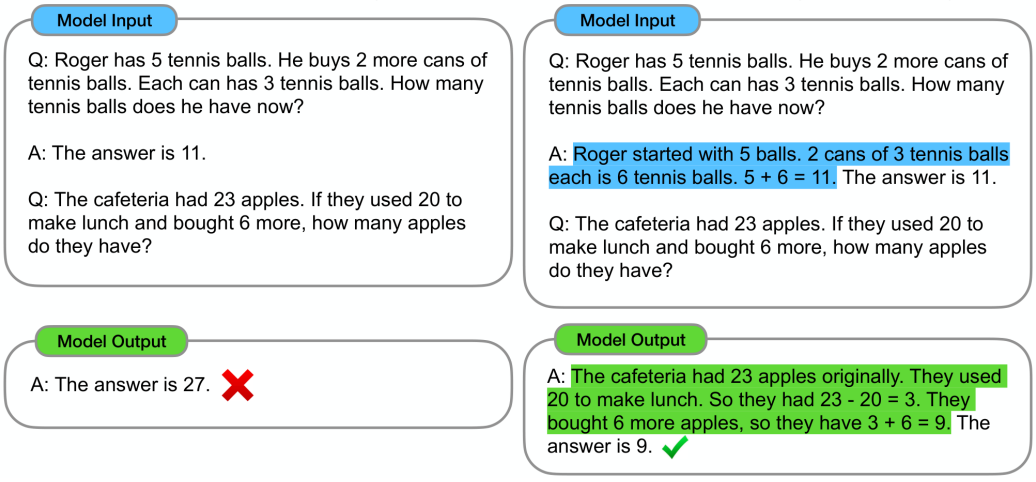
- **형식 일관성 유지**
- **1∼4개가 일반**: 대부분의 작업에서 1∼4개 예시로 충분.

Chain-of-Thought: “단계별로 생각해봐”

- 최종 답을 **한 번에** 예측하게 하면

$$P(\text{최종답} \mid \text{질문})$$

  — 중간 단계가 하나도 없는 **어려운** 조건부 확률을 출력해야 함.
- **어려운 확률이 쉬운 조건부 확률의 곱으로 분해**



Wei et al. — “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models” (2022), Figure 1: 표준 프롬프팅(왼쪽, 최종 답을 바로 냄 → 오답)과 CoT 프롬프팅 (오른쪽, 중간 추론을 먼저 씀 → 정답).

CoT의 메커니즘: 숫자로

직접 한 방에 답

- 중간 결과를 조건으로 쓸 수 없어, 각 부분 추론을 **한 번에** 맞히기(하나당 0.5):

$$0.5^3 = 0.125$$

CoT로 단계별로

- 각 중간 결과가 다음 단계의 **조건**이 되어 하나당 0.9:

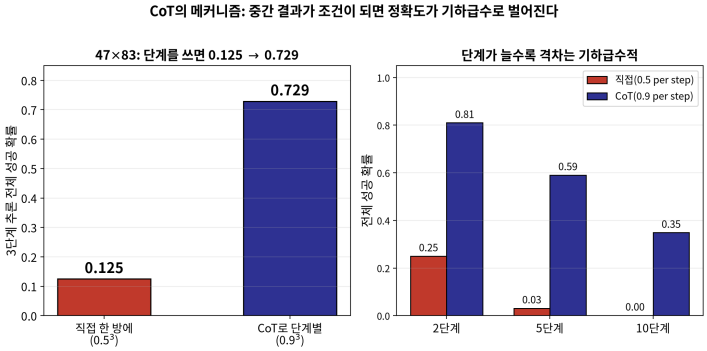
$$0.9^3 = 0.729$$

핵심 메커니즘이 숫자로

모델이 스스로 쓴 중간 결과가, 다음 토큰 예측의 조건이 된다 — 쓰느냐 마느냐가 부분 추론 하나당 0.5에서 0.9로 끌어올린다.

단계가 늘수록 격차 기하급수

그림으로: 단계별로 쓰면 왜 정확해지는가



왼쪽: 3단계 성공 확률(직접 0.125 vs CoT 0.729). 오른쪽: 단계가 늘수록 격차는 **기하급수적**(10단계 0.35 vs 0.001).

손으로: 47×83

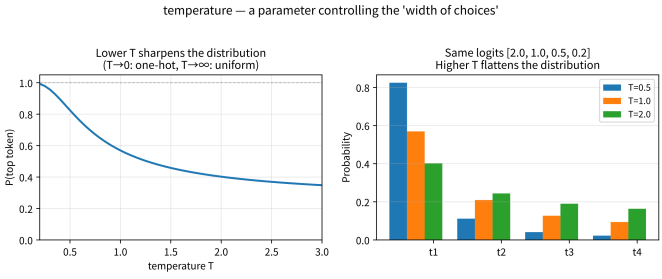
- **CoT** 프롬프트(“단계별로 계산해줘”):
  $47 \times 83 = 47 \times (80 + 3) = 47 \times 80 + 47 \times 3 = 3760 + 141 = \mathbf{3901}$

생성 파라미터: temperature, top-k, top-p

- **Greedy: 확률 최고 토큰 선택**
- **Temperature: 분포 가파름 조절**
- **top-k: 상위 k개 토큰 샘플링**
- **top-p: 누적 확률 p 도달 집합 샘플링**

온도 체감: logits $[2.0, 1.0, 0.5, 0.2]$

T	분포	최상위 확률
0.5	(0.825, 0.112, 0.041, 0.023)	0.825 — 매우 집중
1.0	(0.569, 0.209, 0.127, 0.094)	0.569
2.0	(0.402, 0.244, 0.190, 0.164)	0.402 — 훨씬 평평



같은 logits에 $T \in [0.2, 3.0]$ 적용 시 최상위 확률 곡선(좌: $T \to 0$일수록 1로 수렴)과 $T = 0.5/1.0/2.0$ 분포 막대(우).

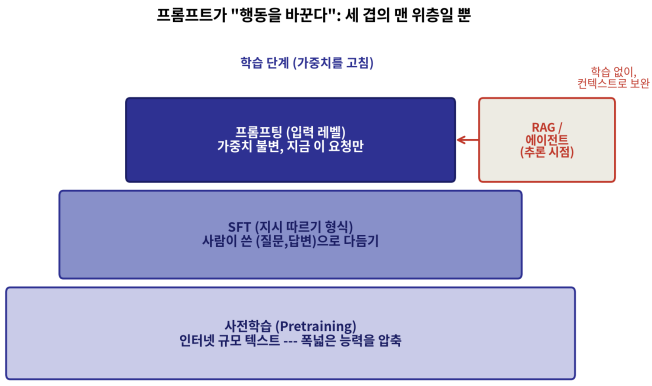
실무 감각

- **결정적** 작업(사실 확인·추출·코드): T 낮게(0∼0.3)
- **창의적** 작업(시·카피·브레인스토밍): T 높게(0.7∼1.0 이상)

프롬프트가 “행동을 바꾼다”는 것의 실제 의미

- **프롬프팅은 입력 레벨의 기법**
- **RAG**(Retrieval-Augmented Generation, Lewis et al., 2020): 질문과 관련된 문서를 **검색해서** 프롬프트에 붙여 “이 문서에 기반해 답하라”.
- **에이전트**(agent, ReAct: Yao et al., 2022): 모델이 검색·계산기·코드 실행 같은 **도구를 스스로 호출**하도록 프롬프트 설계.

그림으로: 세 겹의 관계



사전학습(폭넓은 능력) 위에 SFT(지시 따르는 형식)가 쌓이고, 프롬프팅은 그 맨 위층에서 **입력만으로** 무엇을 꺼낼지 가리킨다. RAG/에이전트는 가중치를 건드리지 않는 별도의 **추론 시점** 경로.

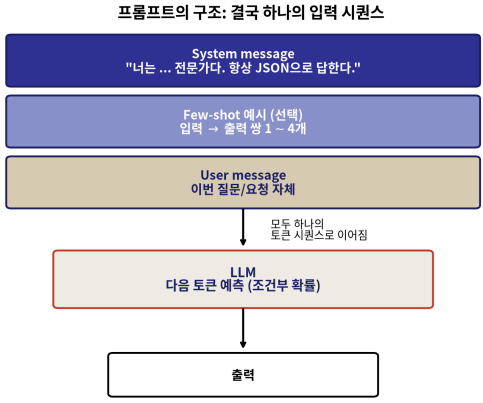
프롬프트의 구조: system / user / few-shot

- **System message: 기본 전제**
- **User message**(질문): 이번 요청 자체.
- **Few-shot 예시**(선택): 질문 앞에 붙는 입출력 쌍.

마법이 아니라 다음 토큰 예측

이 구조 자체는 Week 12의 "입력 시퀀스"일 뿐 — 모델은 system/user를 **구분해서** 처리하는 것이 아니라, **모든 토큰을 같은 다음 토큰 예측 조건으로** 취급. "역할"이 작동하는 것도 그 텍스트가 **조건에 포함**되기 때문. (그래서 system message를 "지키지 않으면"이라는 표현이 애매 — 모델에게는 그저 긴 컨텍스트의 앞부분.)

그림으로: 하나의 시퀀스로 합쳐진다



system · few-shot · user 메시지는 결국 **하나의 입력 토큰 시퀀스**로 이어져 LLM에 들어간다 — 모델은 이 셋을 “구분”해서 처리하는 게 아니라 모두 같은 다음 토큰 예측의 조건으로 취급한다.

환각(Hallucination): 왜 생기는가

- LLM은 환각을 일으킬 수 있다
- 사실 검증 과정 부재

환각의 3가지 유형

환각을 줄이는 실무 수단

- **RAG: 근거 문서를 프롬프트에 포함**
- **Self-consistency: 다수결 채택**

자주 하는 실수 (17.2)

- **“프롬프트 = 질문 한 줄”로 쓰기** — 역할 지정 없으면 모델이 “어떤 식의 번역인가”를 추측. 프롬프트는 **프로그램**: 역할·작업·형식·예시를 명시할수록 출력이 예측 가능.
- **few-shot 예시 형식은 모두 동일해야**
- $T = 0$**은 결정적일 뿐 정확하지는 않음**

확인 문제 (17.2)

- **1 (temperature 손 계산)** 어휘 3개 $\{A, B, C\}$의 logits가 $[3.0, 1.0, 0.0]$. $T = 1$ softmax 분포를 구하고(힌트: $e^3 \approx 20.1, e^1 \approx 2.72, e^0 = 1$), $T \to 0$일 때 분포가 어떤 형태로 수렴하는지 한 문장으로.
- **2 (개념)** 왜 $T > 0$ 샘플링이면 완전히 같은 프롬프트라도 매번 다른 답이 나올 수 있는가 — "다음 토큰 예측이 **확률분포**를 반환한다"는 17.1의 사실로부터 2∼3문장으로 유도. "매번 다른 답"이 **불편하지 않은** / **불편한** 작업 각 하나.
- **3 (코딩)** 장난감 모델로 "OO는 OO이다" 패턴 예시 3개 + "사과는" 질문: zero-shot / two-shot / few-shot 출력을 비교해 "예시 개수가 예측 확률에 어떻게 영향을 미치는가"를 숫자로 확인.

자주 묻는 질문 (17.2)

- **Q. Few-shot 예시는 몇 개?** 정해진 답은 없다. 1∼2개로 충분한 경우도 있고, 복잡할수록 더 많은 예시가 도움. 다만 어텐션 n^2 계산량을 고려해 무한정 늘리는 것은 비용·속도 트레이드오프 — 실무는 1∼4개에서 시작해 "더 늘렸을 때 실제로 오르는지" 확인.
- **Q. CoT가 왜 정확도를 높이나요?** 최종 답을 곧바로 예측하게 하면 중간 추론 없이 "그럴듯한 답"을 내놓아야 하지만, 중간 과정을 먼저 출력하면 그 과정 자체가 **다음 토큰의 조건**이 되어 "참고 자료"로 활용 가능. 0.9^3 vs 0.5^3 숫자 예가 바로 이 메커니즘의 단순 수치 모델.
- **Q. 프롬프팅도 "학습"의 일종인가?** 아니요 — 학습 = 가중치 **갱신**(영구), 프롬프팅 = 가중치 **고정** + 컨텍스트 **조건** 변경(일시, 그 요청 한 번). "**in**-context"의 in = **컨텍스트 윈도우 안에서**. 프롬프트를 빼면 **완벽하게** 원래대로.
- **Q. zero-shot이 항상 열등인가?** 아니요 — **흔한** 작업(번역, 요약, 일반 분류)에서는 사전학습에서 수십억 번 봤기에 충분. few-shot이 특히 유리한 경우: (a) **비표준 형식**, (b) **다의적 라벨**, (c) **새로운 도메인**. 예시를 **너무 많이** 넣으면 (i) n^2 비용, (ii) **예시 편향** 전이. **"더 많은 예시 = 더 좋아"는 항상 성립하지 않는다.**
- **Q. "사실이 아니야, 틀리지 마"라고 쓰면 환각이 줄까?** 약간은 줄 수 있다(더 신중하게 라는 형식에 SFT됐을 수 있음). 하지만 **보장은 아님** — 모델은 사실 검증기가 아니라 **그럴듯한 토큰 생성기**. 환각을 **구조적으로** 줄이려면 근거 공급(RAG)·다수결(self-consistency)·**외부 검증**을 프롬프트 바깥에서 덧붙이는 것이 훨씬 효과적.

실습 노트북 (17.2)

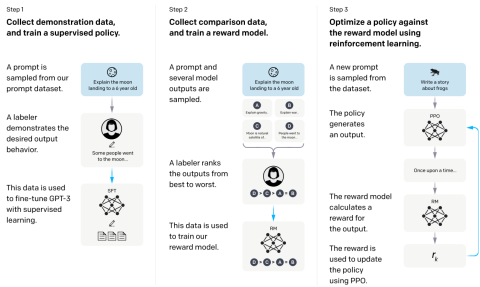
- 17.1의 `next_token_distribution` 구현과 **zero-shot vs few-shot** 출력 대조 (본문 코드 블록 재현).
- `temperature`를 붙인 장난감 모델 — 같은 "사과는" 프롬프트에서 $T = 1.0$ vs $T = 3.0$의 20회 샘플링 결과 비교(같은 프롬프트에서 왜 매번 다른 답이 나오는가).
- **temperature 곡선** — logits $[2.0, 1.0, 0.5, 0.2]$에 $T \in [0.2, 3.0]$ 적용 시 최상위 확률의 변화(본문 온도 그림).
- **CoT vs 직접 답 확률 시뮬레이션** — 3단계 추론, 부분 추론 성공률(중간 결과 조건으로 쓸 때 0.9 vs 없으면 0.5), 4,000회 반복: 직접 $\approx 0.5^3 = 0.125$, CoT $\approx 0.9^3 = 0.729$.

프롬프팅과 추론을 더 깊이: Stanford CS224N(Natural Language Processing with Deep Learning).

17.3 포스트트레이닝: SFT, RLHF, DPO

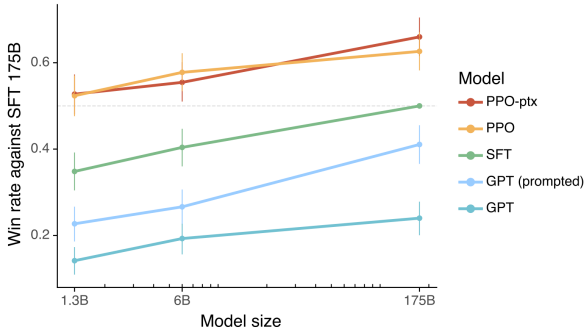
사전학습만으로는 부족한 이유: InstructGPT

- 2022년 OpenAI **InstructGPT**(Ouyang et al., 2022) 놀라운 관찰: 파라미터가 **100배 더 적은** 모델이라도 사람이 원하는 방향으로 다듬어졌다면, 사전학습만 거친 거대 모델보다 사람들이 **훨씬 더 선호**.
- 17.1의 사전학습(다음 토큰 예측)은 "**그럴듯한 텍스트**"를 만들어내지만, 그게 "사람이 원하는 대답"이라는 보장은 아님.
- 사전학습 데이터에는 훌륭한 글도, 엉뚱하거나 무례한 글도 **섞여** 있기 때문.
- **포스트트레이닝**: 사전학습된 모델을 "사람이 실제로 원하는 방향"으로 다듬는 단계.



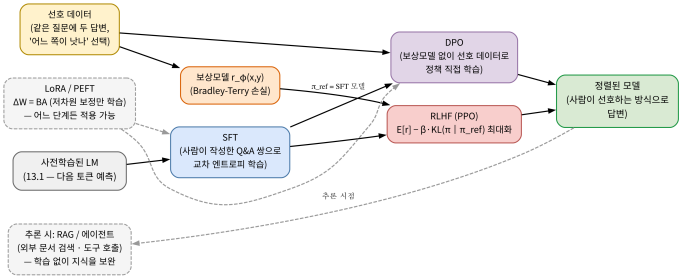
InstructGPT 3단계 학습 파이프라인(원 논문 Figure 2) — (1) SFT, (2) 보상모델(RM) 학습, (3) 이 보상모델을 이용한 PPO 강화학습.

그림으로: 100배 작은 모델이 이긴다



Ouyang et al. — InstructGPT (2022), Figure 1: 175B GPT-3(점선) 대비 각 모델의 인간 선호 승률. **1.3B** InstructGPT조차 **175B** GPT-3보다 선호됨 — 파라미터 수가 아니라 **포스트트레이닝**이 만든 차이.

17.3의 지도: 왜 정렬이 필요한가



- **SFT와 선호 데이터로 두 갈래 정렬**

  학습 단계와 추론 단계 구별

SFT: “이상적인 답변”을 그대로 흉내 내기

- **SFT: 사람이 쓴 고품질 예시로 다음 토큰 예측**
- **답변 부분의 토큰에만** 교차 엔트로피 부과:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{i}\sum_{t} \log \pi_\theta(y^*_{i,t} \mid x_i, y^*_{i,<t})$$

손으로 한 번: SFT가 분포를 어떻게 바꿀까

- 어휘 7개 $\{A_1, \ldots, A_7\}$, 7단계 이상적인 답변. 각 단계에서 정답 토큰에 주는 확률:
  - 사전학습(기준) 모델: $(0.2, 0.5, 0.3, 0.4, 0.3, 0.5, 0.6)$
  - SFT 이후 모델: $(0.8, 0.7, 0.5, 0.6, 0.4, 0.6, 0.8)$
- 사슬 분해 — 이상적 답변 전체 확률 = 각 단계의 곱:

$$P_{\text{ref}}(y^*) = 0.2 \times 0.5 \times 0.3 \times 0.4 \times 0.3 \times 0.5 \times 0.6 \approx 0.0011$$

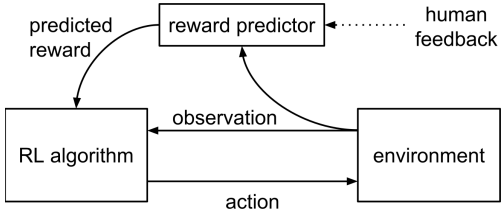
$$P_{\text{SFT}}(y^*) = 0.8 \times 0.7 \times 0.5 \times 0.6 \times 0.4 \times 0.6 \times 0.8 \approx 0.0323$$

핵심: "각 단계는 0.2에서 0.8로만 올랐는데"

곱으로 쌓이므로 짧은 답변에서도 큰 격차 — 17.1의 사슬 분해가 포스트트레이닝에서 다시 작동하는 모습. (정확한 값은 실습 노트북 Section 1에서 검증.)

RLHF: 작성 대신 비교로

- **RLHF**(Reinforcement Learning from Human Feedback, Christiano et al., 2017): 다른 전략.
- 사람이 매번 “정답”을 새로 쓰는 대신, 모델이 만든 **여러 답변 중 어느 쪽이 더 나은지 고르기만** 하면 됨 — 비교가 작성보다 훨씬 쉽고 빠름.
- 이 **선호 데이터**로 **보상모델** (reward model)을 학습.
- 그 보상모델의 점수를 **최대화**하도록 언어모델 자체를 조정.



RLHF 구조(원 논문 Figure 1) — 보상 예측기(reward predictor)가 트래젝터리 세그먼트의 인간 비교 피드백으로 비동기 학습된 뒤, 그 보상을 이용해 정책을 강화학습으로 학습하는 파이프라인.

1단계 — 보상모델: Bradley-Terry 모델

- **브래들리-테리**(Bradley-Terry) 모델 — 선호 확률을 시그모이드로:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$$

왜 시그모이드인가 (Week 2와 같은 이유)

(1) 점수 차이 $\in \mathbb{R}$을 [0,1] 확률로 매핑, (2) 점수 차이가 선호 확률의 **로그-odds**가 선형 스케일로: 차이 +1 $\rightarrow$ odds e배, +2 $\rightarrow$ e^2배 (17.1 softmax의 "원점수 차이 = 확률 비율"과 동일 구조), (3) 극단적 차이에서도 손실이 0에 **유한하게** 접근($-\log\sigma(z) \sim e^{-z}$) $\rightarrow$ 잘 구분하는 쌍이 학습을 지배하지 않음.

BT 손실: 코드와 숫자

```python
import math

def sigmoid(z):
    return 1 / (1 + math.exp(-z))

def reward_model_loss(r_win, r_lose):
    # Bradley-Terry: P(y_w > y_l) = sigmoid(r_win - r_lose)
    return -math.log(sigmoid(r_win - r_lose))
```

- **손으로:** $r_w = 1$, $r_l = -1$ **(margin** $z = 2$**)** $\Rightarrow$

손으로: 2차원 보상모델을 실제로 학습

- **BT 손실은 로지스틱회귀와 같은 모양**

질문	winner (길이, 거절)	loser (길이, 거절)
Q1	(2, 0)	(5, 0)
Q2	(3, 0)	(7, 1)
Q3	(4, 0)	(8, 2)
Q4	(2, 0)	(3, 1)
Q5	(5, 0)	(4, 1)

보상모델은 “점수”가 아니라 “상대 순서”

- 위 장난감 실험이 보여주는 것: 보상모델이 “**점수**를 만드는 것”이 아니라 **“상대 비교”의 일관된 순서를 만드는 것**.
- r_ϕ에 상수를 더해도(모든 점수 +100) 모든 margin이 **불변** → BT 손실도 **불변**. ⇒ 점수는 **절대값에 의미가 없고 차이(margin)에만** 의미가 있다.
- **2단계 — 강화학습으로 정책 학습**: LM을 “질문 x라는 **상태**에서, 답변 y라는 **행동**을 **토큰 하나씩** 순차적으로 고르는 **정책**”으로 취급.
- 보상모델 점수 $r_\phi(x, y)$를 보상 삼아 **강화학습**(상태·행동·보상; 이번 학기 주제 아님, ML2 전문)으로 파라미터 조정.

왜 “토큰 하나씩”이 강화학습인가

목표 함수의 관점에서 전혀 다른 문제: SFT는 정답 토큰이 주어지므로 각 토큰 손실을 **독립**으로 계산 가능. RLHF는 정답이 **없다** — 보상모델은 **완성된 답변 전체**에 하나의 점수만 줄 뿐. “높게 점수 받은 **공로**가 어느 토큰에 있는가” (credit assignment)를 가르는 것이 **강화학습이 존재하는 이유**.

17.2 CoT와 대비: CoT는 중간 결과를 출력물에 써서 다음 토큰 예측의 **조건**으로(추론 시점), RLHF는 출력 전체의 보상을 출력 경로의 확률 (로그-우도)에 비례해 **역분배**(학습 시점) — “프롬프팅은 즉각적이고 파인튜닝은 영구적”의 재확인.

- 보상모델은 수천∼수만 개 (질문, 답변-답변) 쌍에서 학습 → Week 6의 **과적합**에 그대로 노출.
- 훈련 데이터에서 일관된 선호를 보이는 특정 **형식**(특정 어미, 특정 길이)을 **지나치게** 높은 점수로 매기면 → 정책이 그 형식만 반복하는 “**보상 해킹의 특화판**” — 형식은 맞는데 **내용은 공허**한 답변.
- 실무 완화 장치:
  - 보상모델을 **여러 개로 학습해 평균**
  - **KL 항을 동적으로 조절**

  — 모두 이 과적합을 완화하기 위함.

PPO: 목적함수의 형태

- 앞항: 보상모델 점수의 기대를 최대화. 뒤항: **기준 모델**(사전학습 + SFT 후의 π_{ref}) 에서 벗어나는 정도에 β로 **벌**.
- **KL 수치의 감각**: 2.3절의 $D_{KL}(p\|q) = \sum p \log(p/q)$. 두 토큰 어휘 $\{A, B\}$, $\pi_{\text{ref}} = (0.5, 0.5)$라 하면:
  - A에 몰아붙인 극단 $\pi = (0.95, 0.05)$:

$$D_{KL} = 0.95 \cdot \log(0.95/0.5) + 0.05 \cdot \log(0.05/0.5) \approx 0.610 - 0.115 \approx 0.495$$

  $\beta = 1$이면 이 이탈에 0.5의 **벌**.

- 업데이트 후 정책이 이전 정책의 $\frac{\pi_\theta}{\pi_{\text{old}}} \in [1-\epsilon, 1+\epsilon]$(보통 $\epsilon = 0.2$)을 벗어나면 그 영역에서 목적함수의 기울기를 0으로 만듦.

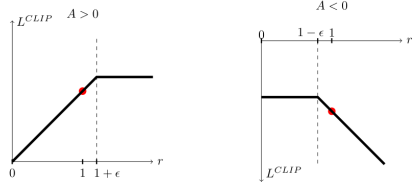


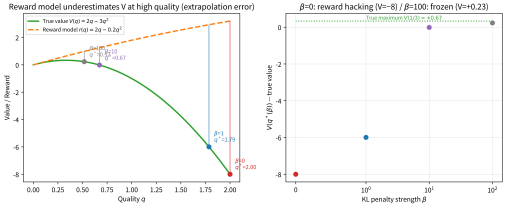
Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r, for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

PPO 클리핑 목적함수 L_{CLIP} 한 항(단일 timestep)을 확률비 r의 함수로 — 좌: 이익 $A > 0$, 우: $A < 0$(원 논문 Figure 1).

보상 해킹을 숫자로: KL 항 없으면

- **보상모델은 고품질 구간에서 실제 가치 하락을 과소평가**

β	q^*	보상 $r(q^*)$	KL	진짜 가치 $V(q^*)$
0	**2.00**	3.20	1.125	$-$**8.00**
1	1.79	2.93	0.827	-5.99
10	0.67	1.26	0.015	-0.01
100	0.52	0.98	0.0002	**+0.23**
(V 진짜 최대)	0.33	—	—	**+0.67**



$\beta = 0$: $q^* = 2.00$까지 밀려나 $V = -8.00$ /
$\beta = 100$: $q^* = 0.52$ 동결 ($V = +0.23$) / 진짜 최대 $q = 1/3$ ($V = +0.67$).

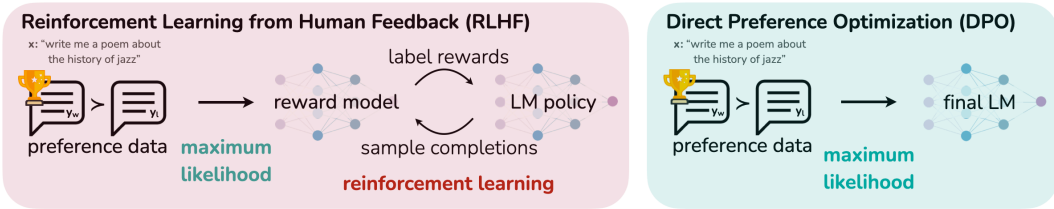
- $\beta = 0$: **경계** $q = 2$까지 밀려 보상 점수 3.20을 얻지만 **진짜 가치는 -8** — 보상 점수는 계속 오르는 데 실제 품질은 무참히 추락. 전형적 **보상 해킹**(Gao et al., 2022).
- $\beta = 10$: $q^* \approx 0.67$, KL ≈ 0.015 거의 0 — 하지만 V의 진짜 최대($q = 0.33, +0.67$)는 여전히 놓침.
  $\beta = 100$: 거의 π_{ref} 동결 — **안정적이지만 학습 효과도 거의 없음**.

DPO: 보상모델도 RL 루프도 없이

- **보상모델도 강화학습 루프도 없이**

핵심

형태를 보면 여전히 위의 보상모델 손실과 **똑같은 로지스틱 손실** — 다만 "보상"의 역할을 $\beta \log(\pi_\theta / \pi_{\text{ref}})$(정책이 기준 모델보다 그 답변을 얼마나 더 선호하게 됐는가)가 대신한다. **RLHF의 2단계 과정 전체를, 지도학습 손실 하나로 대체.**



Rafailov et al. — “Direct Preference Optimization” (2023), Figure 1: RLHF(보상모델 학습 → RL, 왼쪽)와 DPO(선호 데이터에서 정책을 직접 최적화, 오른쪽)의 파이프라인 비교.

DPO의 3단계 논리: 왜 π_{ref}의 비율인가

- **최적 정책은 닫힌 형태**
- $Z(x)$ **소거로 비율만 필요**

DPO 핵심: 보상 내장

정확한 그림

"DPO가 항상 낫다"가 아니라 — **RLHF의 2단계를 1단계로 압축하되, 그 2단계의 부가 가치를 포기하는 trade-off**.

숫자로: DPO 손실의 그래디언트 방향

- 17.2 장난감 상황(두 후보 y_w, y_l, $\beta = 1$). 초기: $\pi_\theta(y_w|x) = 0.6$, $\pi_\theta(y_l|x) = 0.2$, $\pi_{\text{ref}}(y_w|x) = 0.4$, $\pi_{\text{ref}}(y_l|x) = 0.3$.
- 두 로그-비율: $\log \frac{0.6}{0.4} \approx 0.4055$, $\log \frac{0.2}{0.3} \approx -0.4055$.
- margin $z = 0.4055 - (-0.4055) \approx 0.811$, 손실 $\mathcal{L} = -\log \sigma(0.811) \approx \mathbf{0.368}$.
- **BT 그래디언트는 y_w 올리고 y_l 내림**

LoRA: W는 고정한 채 $\Delta W = BA$만 학습

- W **고정,** $\Delta W = BA$**만 학습**
- 학습 파라미터 수를 **수백 분의 1**로 줄이면서도 실전에서 전체 파인튜닝과 **비슷한 성능**.
- **LoRA 가설:** ΔW**는 낮은 계수**

rank r	파라미터 수 (vs 전체 400)	상대 오차
4 (정확)	160 (40%)	≈ 0
3	120 (30%)	0.326
1	40 (10%)	0.701

r=8 시 파라미터 0.39%

에이전트와 RAG: “모델이 필요할 때 바깥에 접근”

- **RAG**(Retrieval-Augmented Generation, Lewis et al., 2020): 질문과 관련된 문서를 **먼저 검색해** 프롬프트에 함께 넣어줌.
- **에이전트**(agent, ReAct: Yao et al., 2022): 한 걸음 더 — 모델이 검색·계산기·코드 실행 같은 **도구를 스스로 호출**하도록 만듦.
- 둘 다 “파라미터 안에 모든 지식을 우겨넣는” 대신 “**모델이 필요할 때 바깥 정보/도구에 접근**하게 한다”는 **같은 방향**의 해법.

RAG를 17.2의 언어로

RAG는 **프롬프팅의 확장**: few-shot 예시가 “모델이 **조건**으로 삼는 토큰”을 확장한다면, RAG는 **검색된 문서**를 그 조건으로 삼는 것. 모델의 파라미터는 **아무것도** 바뀌지 않고, **컨텍스트에 문서를 넣는 행위 자체가 학습을 대체**.

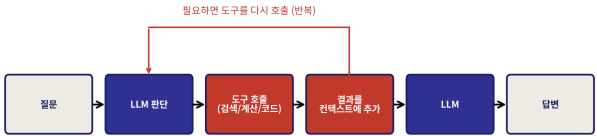
그림으로: RAG와 에이전트의 흐름

에이전트와 RAG: "모델이 필요할 때 바깥에 접근"

RAG: 검색해서 컨텍스트에 넣기



에이전트: 스스로 도구를 호출하고 반복



RAG는 검색한 문서를 프롬프트에 한 번 끼워 넣고 끝나지만, 에이전트는 “도구 호출 → 결과를 컨텍스트에 추가”를 필요한 만큼 **반복**한다 — 둘 다 학습 없이 컨텍스트만으로 동작.

세 방식 비교: SFT vs RLHF vs DPO

	SFT	RLHF	DPO
사람이 주는 데이터	“이 답변이 좋다”(작성)	“이쪽이 더 낫다”(비교)	“이쪽이 더 낫다”(비교)
학습 대상	정책 π_θ	보상모델 r_ϕ + 정책 π_θ	정책 π_θ
손실	다음 토큰 예측 CE	BT(RM) + PPO(RL)	BT(정책 로그-비율)
보상모델	없음	별도 학습	없음(정책이 겸임)
강화학습 루프	없음	있음(불안정, 보상 해킹 위험)	없음
KL 항의 역할	—	보상 해킹 방지(β 튜닝)	β가 같은 역할
상한	사람이 작성할 수 있는 품질	(보상모델의 근사 한계)	(데이터의 선호 순서 한계)
LoRA 적용	가능	가능(RM도, 정책도)	가능

계층도 구분할 것

LoRA = **학습 단계**의 효율화(어느 파인튜닝 단계든 적용 가능) / **RAG/에이전트** = **추론 단계**의 지식 보완(학습 없이 컨텍스트에 넣음)
— **둘을 동시에** 쓰는 것이 실무 표준.

“LoRA 파인튜닝 + RAG 최신 정보 공급 + 에이전트 도구 호출”은 각각 **서로 다른 단계**의 문제(가중치 비용 / 지식 시효 / 계산 능력)를 해결하는 **독립 축**.

17.3 자주 하는 실수

1. **SFT 데이터 품질이 수량보다 중요**

1. **DPO 손실 계산**
2. **PPO 목적함수 분석**
3. **LoRA 파라미터 계산**

예제: DPO 손실(장난감)

```python
import numpy as np
from scipy.special import expit  # logistic(sigmoid)

def dpo_loss(pi_th_w, pi_ref_w, pi_th_l, pi_ref_l, beta=1.0):
    margin = beta * (np.log(pi_th_w / pi_ref_w) - np.log(pi_th_l / pi_ref_l))
    return -np.log(expit(margin))

print(dpo_loss(0.7, 0.4, 0.1, 0.3))        # 0.1739
print(dpo_loss(0.7, 0.4, 0.1, 0.3, beta=0.1))  # 0.6533
print(dpo_loss(0.6, 0.4, 0.2, 0.3))        # 0.3682 초기()
```

17.3 확인 문제

1. SFT 손실 $\mathcal{L} = -\frac{1}{T}\sum \log \pi_\theta(y_t|y_{<t})$에서, SFT의 **입력 데이터**는 (질문, **정답 답변**) 쌍 — 이 "정답 답변"을 누가 작성하는가?
2. RLHF 2단계에서 **보상모델**이 학습받는 손실은? **정책**이 학습받는 목적함수(17.3의 $J(\theta)$)를 쓰고, KL 항이 **왜** 필요한가?
3. DPO 손실을 BT 손실과 **비교**: BT의 $\log r(x, y_w) - \log r(x, y_l)$ 대신 들어가는 항을 쓰고, β가 어떤 역할을 하는가.
4. **판별:** "DPO는 보상모델이 필요 없으므로 RLHF보다 항상 안정적이다" — 참/거짓, 이유(보상 해킹 관점).
5. LoRA rank $r = 8$을 $r = 64$로 올리면: (a) 학습 파라미터 수가 몇 배? (b) **과적합** 위험은 어떻게 변하는가? (c) 17.3의 SVD 근사 결과(rank-4: 0.0%, rank-3: 0.3%, rank-1: 0.7%)와 비교.
6. RAG: 질문과 **검색된 문서**를 컨텍스트에 넣는다 — 17.2의 "컨텍스트 = **조건**" 관점에서 RAG는 **학습**인가, **조건화**인가?